CG 第二次作业实验报告

PB23000052 杨硕

一、实验目的与任务概述

本次作业的主题为 Image Warping。任务目标是在给定若干组控制点对的情况下，构造二维平面上的插值变形映射，使源图像中的指定点被连续地拖拽到目标位置，并将该映射作用到整张图像上。作业要求至少实现两种 warping 方法：Inverse Distance Weighted Interpolation（IDW）与 Radial Basis Function（RBF），同时需要按照面向对象设计思想，将“数学映射”和“图像处理”解耦。

本次实现完成了 IDW 与 RBF 两种 warping 算法，并在图像组件中支持控制点交互、变形计算与结果显示。通过这些模块，可以对图像进行基于控制点的连续拖拽变形。

1.1 作业要求

- 至少实现两种图像变形算法：IDW 与 RBF。
- 使用面向对象的方式封装变形映射，使数学模型独立于图像类。
- 在界面中支持控制点的交互式选取与可视化。

1.2 开发环境

- 语言：C++20
- 框架：Framework2D
- GUI：Dear ImGui
- 图像显示与交互：OpenGL + GLFW + GLAD
- 文档编写：Typst

二、算法原理

2.1 图像变形的数学抽象

设控制点对为

$$(p_i, q_i), i = 1, 2, ..., n$$

，其中

$$p_i \in \mathbb{R}^2$$

表示原图中的控制点，

$$q_i \in \mathbb{R}^2$$

表示其在目标图像中的对应位置。问题可以抽象为构造一个映射：

$$f : \mathbb{R}^2 \to \mathbb{R}^2$$

满足：

$$f(p_i) = q_i, \quad i = 1, 2, ..., n$$

在实际应用中，映射不仅要在控制点处严格满足插值条件，还应在控制点之间保持平滑，并尽量避免局部不连续和整体畸变过大。

2.2 IDW 变形原理

IDW 方法将整体映射表示为多个局部近似映射的加权平均：

$$f(p) = \sum_i w_{i(p)} f_{i(p)}$$

其中权重函数采用反距离形式：

$$w_{i(p)} = \frac{\sigma_{i(p)}}{\sum_j} \sigma_{j(p)}, \quad \sigma_{i(p)} = \frac{1}{\|} p - p_i \|^{\mu}$$

本实现中取

$$\mu = 2$$

。这意味着待变形点越靠近某个控制点，该控制点定义的局部映射对最终结果的影响越大。

每个局部映射采用仿射形式：

$$f_{i(p)} = q_i + T_i(p - p_i)$$

其中矩阵

$$T_i$$

由其余控制点构成的加权最小二乘问题求得。这样可以保证映射在控制点附近既满足插值条件，又具有局部线性近似能力。IDW 的优点是实现直观、局部性强，对少量控制点也有较稳定的拖拽效果；缺点是控制点较多时需要对每个控制点建立局部矩阵，且形变远场行为不如带全局项的模型稳定。

2.3 RBF 变形原理

RBF 方法将变形映射表示为仿射部分与径向基部分之和：

$$f(p) = A(p) + R(p)$$

其中仿射项写为：

$$A(p) = Mp + b$$

径向基部分写为：

$$R(p) = \sum_i \alpha_i g(\|p - p_i\|)$$

这里

$$\alpha_i \in \mathbb{R}^2$$

为待求系数。根据作业文档中的提示，本实现选取

$$g(d) = \sqrt{d^2 + r_i^2}$$

其中

$$r_i$$

由控制点之间的最近邻距离估计得到。实现时，先利用控制点对拟合一个全局 affine 变换，再把控制点的剩余位移作为线性方程右端，求解所有

$$\alpha_i$$

。这样做的好处是：整体大尺度变形由 affine 项承担，而局部细节由 RBF 残差项修正，因此在大范围拖拽时比纯局部模型更平滑。

三、程序设计与实现

3.1 面向对象结构

本次作业将 warping 的数学部分与图像组件分离。整体结构可以概括为两层：

- `Warper` 抽象类：只关心二维点到二维点的映射。
- `WarpingWidget` 图像组件：负责控制点交互、图像遍历与变形结果显示。

这种划分的优点在于：IDW 与 RBF 的差异集中在数学映射上，而图像层的遍历、采样和可视化逻辑可以复用。后续如果增加更多 warping 方法，只需新增一个 `Warper` 子类即可。

3.2 Warper 抽象类

`Warper` 是本次实现的核心抽象。该类定义了统一接口 `warp(point)`，输入一个二维点，输出其变形后的位置。此外，`Warper` 还封装了若干基础数值工具，包括：

- 二维点距离计算；
- 2x2 线性系统求解；
- 一般线性方程组高斯消元求解；
- 根据控制点对拟合 affine 变换。

这些通用工具由 `IDWWarper` 和 `RBFWarper` 共享，避免了重复实现。

3.3 IDWWarper 实现

`IDWWarper` 在初始化时完成两件事：

1. 保存所有控制点对；
2. 对每个控制点建立局部仿射矩阵

$$T_i$$

。

在调用 `warp(p)` 时，程序会先判断 `p` 是否恰好落在某个控制点上；若是，则直接返回对应目标点，保证插值条件严格成立。否则，对每个局部映射计算权重并进行归一化，再将各局部结果加权平均得到最终变形位置。

这种设计与文档中的数学定义一一对应，且逻辑清晰。

3.4 RBFWarper 实现

RBFWarper 的初始化过程包括：

1. 使用控制点对拟合 affine 变换；
2. 根据控制点之间的距离估计每个基函数的尺度参数；
3. 构造 RBF 线性系统并分别求解 x、y 两个方向的系数。

在 warp(p) 时，先计算 affine 项，再叠加所有径向基函数的贡献。由于 RBF 天然具有全局支撑特性，控制点的移动会对整个图像产生连续影响，因此图像形变通常较为平滑。

3.5 图像 warping 流程

图像层采用如下处理流程：

1. 用户在界面中拖拽，记录控制点对 start_points_ 与 end_points_；
2. 根据当前算法类型创建 IDWWarper 或 RBFWarper；
3. 遍历原图每个像素，计算其变形后的目标坐标；
4. 根据映射结果将原图颜色写入目标图像；
5. 更新图像纹理并刷新窗口显示。

这里采用基于控制点定义的正向变形映射来驱动整张图像的形变，使图像编辑过程与文档中的数学模型保持一致。

四、实验结果与分析

4.1 交互界面

程序运行后，界面顶部提供 Select Points、Warping、Restore 等菜单项，并可切换 Fisheye、IDW 与 RBF 模式。用户在图像上拖拽时，会显示蓝色起点、绿色终点和红色连接线，便于观察控制约束。

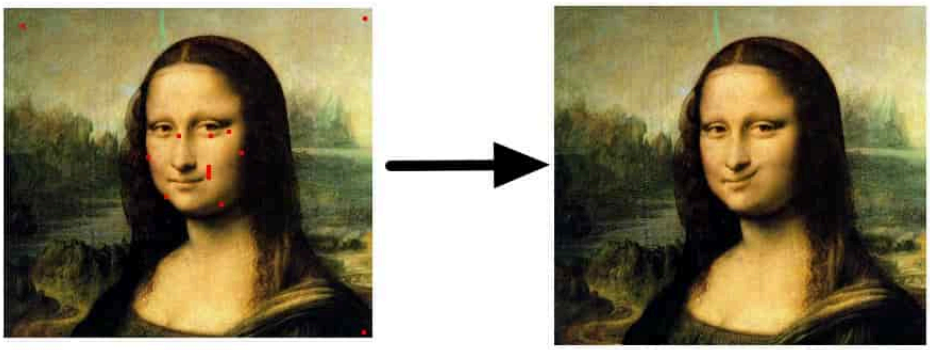

Figure 1: Homework 2 界面示意

4.2 IDW 与 RBF 的效果比较

在控制点数量较少时，IDW 能快速得到较自然的局部拖拽效果，受影响区域主要集中在控制点附近，适合做小范围局部编辑。RBF 则具有更明显的全局平滑特性，当控制点分布较分散或位移较大时，整体形变往往更连续。

从主观观察上看：

- IDW：局部性更强，控制点附近响应更直接；
- RBF：全局过渡更平滑，但远处区域也会被一定程度拉动；
- 对大范围拖拽，RBF 的整体连续性通常更好；
- 对精细局部编辑，IDW 更容易体现“哪里拖哪里动”的直观感觉。

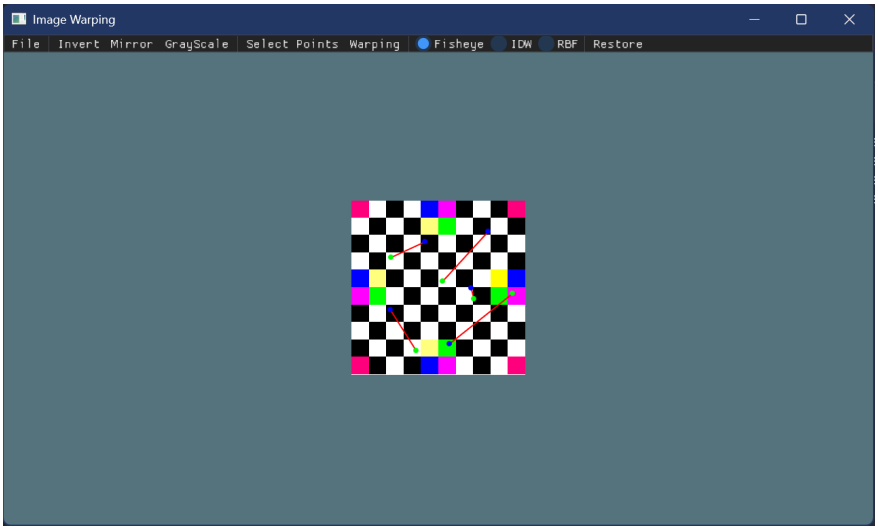

Figure 2: 控制点拖拽示意

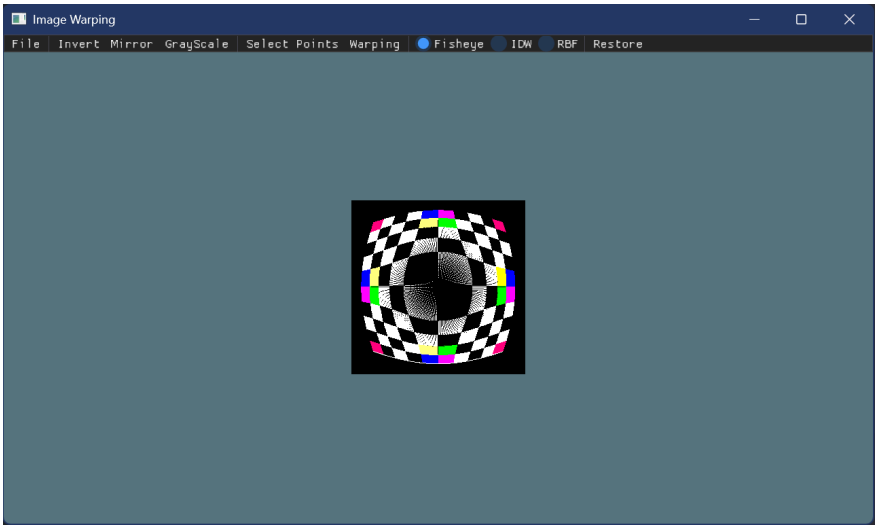

Figure 3: Warping 结果示意

4.3 实现中的问题与改进

本次实现仍有一些可继续改进的地方：

- RBF 实现采用的是 affine 项加残差拟合的形式，若进一步加入完整 side constraints，可获得更标准的块矩阵解法；
- 若采用 backward mapping 并显式数值求逆，也有机会进一步提高采样精度，但实现复杂度会明显增加。

五、总结

本次作业围绕 Image Warping 的数学建模与工程实现展开，完成了 IDW 与 RBF 两种图像变形算法，并使用面向对象方式将 warping 映射从图像处理逻辑中解耦。

通过本次作业，我对以下内容有了更直观的理解：

- 控制点插值如何扩展为整张图像的连续变形映射；
- IDW 与 RBF 在局部性、平滑性和远场行为上的差异；
- 面向对象设计如何帮助组织图形学程序中的数学模块和交互模块；

整体而言，这次作业将“数学插值”“图像采样”和“交互系统”结合在了一起，是一次兼具理论性与工程性的练习。